CS 311: Object Oriented Programming (1)
Final Project Report Template

PROJECT TITLE: LIBRA MANAGEMENT SYSTEM

MEMBERS:

	Name	ID	Role Participated in	Sign
1	Haneen Abdulaziz Mtashsh	[redacted]	Programmer, Leader	
2	[redacted]	[redacted]	Programmer, Documentation Specialist	
3	[redacted]	[redacted]	Programmer, Tester	
4	[redacted]	[redacted]	Programmer, Designer	
5	[redacted]	[redacted]	Programmer, Tester	
	[redacted]	[redacted]	Programmer, Documentation Specialist	

DIFFICULTIES & HOW PROBLEMS WERE SOLVED:

Code Combining Issue :
Because of variances in variable and method names, team members' code fails to work when combined.
Solution: Organize a team meeting to evaluate each class's functional tasks and methods. Before combining code segments, make sure they all work independently.

Data Loss Issue :
When the program is closed, all modifications are lost because the data was not saved in a permanent location like a file.
Solution:
Save the data : When any modification is made (such as deleting or adding a book), update the file and save the data into it. When the program starts, it will read the data from the file.

Difficulty Understanding Written Code :
We found it difficult to understand the code written by other team members.
Solution:
Each member should add comments to their code to clarify the purpose of each part.

CHECKLIST OF REQUIREMENTS FULFILLED IN THE PROJECT:

Function / Application Aspect	UML Diagrams	Classes Management	Inheritance & Polymorphism	ArrayList	Array Of Objects	Program Structure	Error Handling	Analysis
• User superclass: Customer and Admin subclasses. • Book superclass: BorrowedBooks and BoughtBook subclasses • Payment superclass : Card and Cash payment subclasses	✓	✓	✓		✓	✓		
Customer browsing books		✓	✓		✓	✓		
Customer borrowing books		✓	✓	✓	✓	✓	✓	Cart (array) of books (copy array items)
Customer buying books		✓	✓	✓	✓	✓	✓	Cart (array) of books (copy array items)
Admin managing books (add, delete, update)	✓	✓	✓	✓	✓	✓	✓	Array of books (array operations such as shifting)
Customer returning books			✓	✓	✓	✓	✓	Cart (array) of borrowed books
Library Statistics (most borrowed books)			✓		✓	✓		Array of most borrowed books

How Were The Tasks Distributed Equally Between Members?

The tasks were distributed equally among the members, with each student assigned her own subclass and its implementation in the main program. Additionally, pairs of students collaborated on superclasses to ensure consistency and integration. Below is a table detailing the tasks completed by each member.

	Name	Classes	Main methods
1	Haneen Abdulaziz Mtashsh	User,Costumer classes	Login , sign up
2	[redacted]	Book, BoughtBook classes	add, displayBooks
3	[redacted]	Payment, CashPayment classes	BuyBook, Remove books
4	[redacted]	Book, BorrowedBooks class	BorrowBook, returnBook
5	[redacted]	Payment, CardPayment class	Initialize books, most borrowedBoughtBooks
6	[redacted]	Admin class	Initialize admin and updatebooks

General Comments:

Working on the Library Management System was a great learning experience for our team. It helped us apply object-oriented programming concepts in a practical way while improving our teamwork and problem-solving skills. We're proud of what we achieved and how much we've learned through this project.

The Updated UML DIAGRAM:

UML diagram

PROJECT DETAILS:

1.CLASSES

```java
public class books {
    private String Title;
    private String Auther;
    private String publisher;
    private String isbn;
    private int numOfCopy;
    private double price;

    books(){

        this(null,null,null,null,0,0.0);
    }
    books(String Title, String Auther, String publisher, String isbn, int numOfCopy, double price){
        this.Title = Title;
        this.Auther = Auther;
        this.publisher = publisher;
        this.isbn = isbn;
        this.numOfCopy = numOfCopy;
        this.price = price;
    }
    public void display(){
        System.out.print("\nBook title: " + Title + "\n" +
                "Book Auther: " + Auther + "\n" +
                "Book publisher: " + publisher + "\n" +
                "Book ISBN: " + isbn + "\n" +
                "Book Copies: " + numOfCopy +"\n" +
                "Book price: " + price);
    }
```

```java
    public String getTitle(){
        return Title;
    }
    public void setTitle(String Title){
        this.Title = Title;
    }
    public String getAuther(){
        return Auther;
    }
    public void setAuther(String Auther){
        this.Auther = Auther;
    }
    public String getpublisher(){
        return publisher;
    }
    public void setpublisher(String publisher){
        this.publisher = publisher;
    }
    public String getisbn(){
        return isbn;
    }
    public void setisbn(String isbn){
        this.isbn = isbn;
    }
    public double getprice(){
        return price;
    }
```

```java
    }
    public void setprice(double price){
        this.price = price;
    }
    public int getCopies(){
        return numOfCopy;
    }
    public void setCopies(int numOfCopy){
            if (numOfCopy >= 0) {
            this.numOfCopy = numOfCopy;
        } else {
            System.out.println("Number of copies cannot be negative.");
        }
    }
    public boolean Availability(){
        if (numOfCopy > 0)
            return true;
        else
            return false;
    }
    public void decrementCopies() {
        if (numOfCopy > 0) {
            numOfCopy--;
        } else {
            System.out.println("No copies available to borrow.");
        }
    }
}
```

The books class handles everything related to the library's collection of books. It stores important details about each book, like the title, author, publisher, ISBN, the number of copies available, and the price. Think of it as the foundation for anything book-related in the system. Whether you're adding new books to the library, updating existing ones, or tracking inventory, this class is where all that happens. It also sets the stage for more specialized classes like borrowed or purchased books.

```java
public class BorrowedBooks extends books {
    private String borrowerName;
    private Date borrowDate ;
    private Date [redacted];

    // Constructer for BorrowBooks
    public BorrowedBooks(String Title,String Auther,String publisher, String isbn,
            int numOfCopy, String borrowerName, Date borrowDate, Date returnDate){
        super(Title,Auther,publisher,isbn,numOfCopy,0);
        this.borrowerName = borrowerName;
        this.borrowDate = borrowDate;
        Calendar calendar = Calendar.getInstance(); // to now the borrow,retune date
        calendar.setTime(borrowDate);
        calendar.add(Calendar.DAY_OF_MONTH, 15); // setting 15 days as a due date
        this.[redacted] = calendar.getTime();
        }
    public String getBorrowerName(){ // getter & setter for borrower name
        return borrowerName;
    }
    public void setBorrowerName(String borrowerName){
        this.borrowerName = borrowerName;
    }
    public Date getBorrowDate() { // getter & setter for borrow date
        return borrowDate;
    }
    public void setBorrowDate(Date borrowDate){
        this.borrowDate = borrowDate;
```

```java
    public Date getReturnDate() {  // getter & setter for return date
        return [redacted];
    }
    public void setReturnDate(Date returnDate){
        this.[redacted] = returnDate;
    }
    public boolean checkIfAvaliable() {
    if (super.Availability()) {
        System.out.print("The book is available for borrowing");
        return true;
    } else {
        System.out.print("Sorry, the book is not available");
        return false;
    }
}

    public boolean lateReturn(Date returnDate) { // method to check the return status
        return returnDate.after(returnDate);
        }
    public void display(){ // displaying book,borrower information
        super.display();
        System.out.println("\n The borrower name is:" + borrowerName);
        System.out.println("The return date is: " + [redacted]);
    }
    public double calculateFine(Date returnDate) {
    long diff = returnDate.getTime() - this.[redacted].getTime();
    long daysLate = diff / (1000 * 60 * 60 * 24); // Convert milliseconds to days
```

```java
    }
    public double calculateFine(Date returnDate) {
    long diff = returnDate.getTime() - this.[redacted].getTime();
    long daysLate = diff / (1000 * 60 * 60 * 24); // Convert milliseconds to days

    if (daysLate > 0) {
        double fine = 20; // Initial fine
        long weeksLate = daysLate / 7;
        fine += weeksLate * 10; // Additional fine for each week
        return fine;
    }
    return 0; // No fine
}
}
```

The BorrowedBooks class extends the books class to manage library books that are borrowed. It tracks the borrower's name, borrow date, and return date, automatically setting the due date to 15 days after borrowing. The class includes methods to check book availability, determine if a return is late, calculate fines for overdue books (starting at 20 units with additional charges for late weeks), and display detailed information about the borrowed book and borrower. It simplifies borrowing operations while ensuring overdue policies are enforced.

```java
public class BoughtBook extends books{
    private Date purchaseDate;
    private String buyerName;
    private int buyerAge;
    private String id;
    private String paymentType;

    // Default constructor
    public BoughtBook() {
        this.purchaseDate = new Date();
        this.buyerName = null;
        this.buyerAge = 0;
        this.id = null;
        this.paymentType = null;
    }
    // Parameterized constructor
    public BoughtBook(String title, String author, String publisher, String isbn,
                      int numOfCopy, double price, String buyerName, int buyerAge,
                      String id, String paymentType) {
        super(title, author, publisher, isbn, numOfCopy, price);
        this.purchaseDate = new Date();
        this.buyerName = buyerName;
        this.buyerAge = buyerAge;
        this.id = id;
        this.paymentType = paymentType;
    }
```

```java
    // Getters and Setters
    public Date getPurchaseDate() {
        return purchaseDate;
    }
    public String getBuyerName() {
        return buyerName;
    }
    public void setBuyerName(String buyerName) {
        this.buyerName = buyerName;
    }
    public int getBuyerAge() {
        return buyerAge;
    }
    public void setBuyerAge(int buyerAge) {
        this.buyerAge = buyerAge;
    }
    public String getId() {
        return id;
    }
    public void setId(String id) {
        this.id = id;
    }
    public String getPaymentType() {
        return paymentType;
    }
    public void setPaymentType(String paymentType) {
        this.paymentType = paymentType;
```

```java
    }
    public void setPaymentType(String paymentType) {
        this.paymentType = paymentType;
    }
    // Method to get the book object
    public books getBook() {
        return this; // Return the current instance of BoughtBook
    }
    // Method to process the book purchase
    public void bookPurchaseProcess(Payment payment) {
        if (super.Availability()) {
            System.out.print("# Book And Buyer Details #");
            super.display(); // Display book details
            System.out.printf("\nBuyer Name: " + buyerName + "\n" +
                    "Buyer Age: " + buyerAge + "\n" +
                    "Buyer National ID: " + id + "\n" +
                    "Payment Type: " + paymentType + "\n" +
                    "Purchase Date: " + purchaseDate + "\n");
            payment.processPayment(); // Call payment processing
        } else {
            System.out.print("Sorry! The book is not available");
        }
    }
}
```

The BoughtBook class builds on the books class to handle book purchases, tracking buyer details like name, age, ID, payment type, and purchase date (set to the current date by default). It offers a parameterized constructor for initializing these details and provides methods to access or update them. The bookPurchaseProcess method checks if the book is available, displays book and buyer information, and processes the payment using the Payment class. If the book isn't available, it notifies the user. This class ensures a smooth purchase experience by managing buyer information and payment processing seamlessly.

```java
public class Payment {
    private double amount;
    private LocalDate paymentDate;

    // Constructor
    public Payment(double amount) {
        this.amount = amount;
        this.paymentDate = LocalDate.now();
    }
    // Getters
    public double getAmount() {
        return this.amount;
    }
    public LocalDate getPaymentDate() {
        return this.paymentDate;
    }
    // Setters
    public void setAmount(double amount) {
        this.amount = amount;
    }
    public void setPaymentDate(LocalDate paymentDate) {
        this.paymentDate = paymentDate;
    }
    // General message that applies to all payments
    public void processPayment() {
        System.out.println("Processing a payment of $" + amount + "  on  " + paymentDate);
        // Add logic for handling payment confirmation, etc.
```

```java
public class CashPayment extends Payment {
    private double cashReceived;
    private double change;

    public CashPayment(double amount, double cashReceived) {
        super(amount);
        this.cashReceived = cashReceived;
        this.change = cashReceived - amount; // Calculates change if cash received is more than total price
    }
    public double getCashReceived() {
        return this.cashReceived;
    }
    public double getChange() {
        return this.change;
    }
    public void setCashReceived(double cashReceived) {
        this.cashReceived = cashReceived;
    }
    public void setChange(double change) {
        this.change = change;
    }
    @Override
    public void processPayment() {
        System.out.println("Processing cash payment of $" + getAmount() + " on " + getPaymentDate());
        System.out.println("Cash received: $" + cashReceived);
        System.out.println("Change: $" + change);
```

The Payment class is designed to manage the core details of any payment transaction within the system. It tracks the payment amount and the payment date, which is automatically set to the current date upon initialization. The class includes getter and setter methods to access and modify these details as needed. Its main functionality is encapsulated in the processPayment method, which outputs a message confirming the payment amount and date. This serves as a foundation for more specialized payment types, like card or cash payments, ensuring that all transactions are recorded and processed consistently.

The CashPayment class extends the Payment class to handle payments made in cash. It introduces attributes for the amount of cash received and the change to be returned, which is calculated automatically when the object is created. The class provides getter and setter methods for these attributes, making it easy to retrieve or update them if necessary. The processPayment method is overridden to include specific details about cash transactions, such as the cash received and the change given. This class ensures efficient handling of cash payments while maintaining clarity and accuracy in transactions.

```java
/
olic class CardPayment extends Payment{
  private String cardNumber;
  private String cardHolderName;
  private String expiryDate;
  private String cvv;

  public CardPayment(double amount) {
      super(amount);
      collectCardDetails();
  }

  // Method to collect card details from the user
  private void collectCardDetails() {
      Scanner scanner = new Scanner(System.in);
      System.out.println("Enter Card Holder Name: ");
      this.cardHolderName = scanner.nextLine();

      System.out.println("Enter Card Number: ");
      this.cardNumber = scanner.nextLine();

      System.out.println("Enter Expiry Date (MM/YY): ");
      this.expiryDate = scanner.nextLine();
      System.out.println("Enter CVV Number: ");
      this.cvv = scanner.nextLine();}

  public String getCardNumber() {
```

```java
  public String getCardNumber() {
      return cardNumber;
  }
  public String getCardHolderName() {
      return cardHolderName;
  }
  public String getExpiryDate() {
      return expiryDate;
  }
  @Override
  public void processPayment(){
      System.out.println("Processing card payment of $"+getAmount() +"on"+getPaymentDate());
      System.out.println("Card Holder: "+ cardHolderName);
      System.out.println("Card Number: "+ "**** **** **** "+ cardNumber.substring(cardNumber.length()-4));
      System.out.println("Expiry Date: "+ expiryDate);
  }
  }
```

The CardPayment class extends the Payment class to manage credit or debit card transactions. It adds attributes for the cardholder's name, card number, expiry date, and CVV. During object creation, the collectCardDetails method prompts the user to input these details. The class provides getter methods for accessing card information, though sensitive details like the card number are displayed partially for security purposes. The processPayment method is overridden to display payment details, including a masked card number and the cardholder's name, while processing the transaction. This class ensures secure and user-friendly handling of card payments within the system.

```java
public class User {
    String name;
    String id;
    String email;
    String phoneNumber;
    int age;

    //constructor
    public User(String name, String id, String email,
            String phoneNumber,int age) {
        this.name = name;
        this.id = id;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.age=age;
    }
    public void displayInfo() {
        System.out.println("Name: " + name +
                           "\nID: " + id +
                           "\nEmail: " + email +
                           "\nPhone Number: " + phoneNumber +
                           "\nAge: " + age );
    }
```

```java
    }
    //display most borrowed/bought books method
    public void mostBorrowed_BoughtBooks(List<BoughtBook> boughtBooks) {
    System.out.println("Most Borrowed/Bought Books:");
    for (BoughtBook book : boughtBooks) {
        if (book.getBuyerName() != null && book.getBuyerName().equals(this.name)) {
            System.out.println(book.getTitle());
        }
    }
}
public void availableBooks(List<books> bookList) {
    System.out.println("Available Books:");
    for (books book : bookList) {
        if (book.Availability()) {
            System.out.println(book.getTitle());
        }
    }
```

```java
}
    public String getName() {
        return name;
    }
    public String getId() {
        return id;
    }
    public String getEmail() {
        return email;
    }
    public String getPhoneNumber() {
        return phoneNumber;
    }
     public int getAge() { // New getter for age
        return age;
    }
    public void setAge(int age) { // New setter for age
        this.age = age;
    }
}
```

The User class represents a library user with essential details like name, ID, email, phone number, and age, which are initialized through the constructor. It provides methods like displayInfo to show the user's personal information and mostBorrowed_BoughtBooks to list the books most frequently borrowed or purchased by the user. The availableBooks method checks and displays books that are currently available in the library. Overall, this class is designed to manage user information and interact with the system's book records, offering a clear and practical way to handle user-related operations.

```java
public class Admin extends User {

//constructor
    public Admin(String name, String id, String email, String phoneNumber,int age){
        super(name, id, email, phoneNumber,age);
    }
    // Method to add a book
    public books addBook(String Title, String Auther, String publisher, String isbn,
            int numOfCopy, double price) {
        books newBook = new books(Title, Auther, publisher, isbn, numOfCopy, price);
        System.out.println("Book '" + Title + "' has been added.");
        return newBook;
    }
    // Method to remove a book
    public void removeBook(books book) {
        if (book != null) {
            System.out.println("Book with ISBN '" + book.getisbn() + "' has been removed.");
            book = null;
        } else {
            System.out.println("No book found.");
        }
    }
    // Method to update a book's details
```

```java
    // Method to update a book's details
    public void updateBook(books book, String newTitle, String newAuther,
            String newPublisher, String newIsbn, int newNumOfCopy, double newPrice) {
        if (book != null) {
            book.setTitle(newTitle);
            book.setAuther(newAuther);
            book.setpublisher(newPublisher);
            book.setisbn(newIsbn);
            book.setCopies(newNumOfCopy);
            book.setprice(newPrice);
            System.out.println("Book has been updated with new details:");
            book.display();
        } else {
            System.out.println("No book found to update.");
        }
    }
}
```

The Admin class extends the User class to represent an administrator with special privileges for managing books in the library. It inherits the user's basic details and adds functionality for managing the library's book collection. The class includes methods to add a new book, remove an existing book, and update a book's details. When adding a book, it creates a new books object and prints a confirmation message. For removing a book, it checks if the book exists and then deletes it, providing feedback to the user. The updateBook method allows an admin to modify a book's details, displaying the updated information. This class ensures that administrators can efficiently manage the library's inventory.

```java
    */
    public class Customer extends User {
        private List<BorrowedBooks> borrowedBooks;

        // Constructor
        public Customer(String name, String id, String email, String phoneNumber,int age) {
            super(name, id, email, phoneNumber,age);
            this.borrowedBooks = new ArrayList<>();
        }

        // Method to borrow a book
        public void borrow(books book) {
            if (book.Availability()) {
                // Create a BorrowedBooks instance
                BorrowedBooks borrowedBook = new BorrowedBooks(
                    book.getTitle(),
                    book.getAuther(),
                    book.getpublisher(),
                    book.getisbn(),
                    book.getCopies(),
                    this.getName(),
                    new Date(), // Borrow date
                    null // Return date will be set later
                );

                // Add to borrowed books
```

```java
                // Add to borrowed books
                borrowedBooks.add(borrowedBook);
                System.out.println(getName() + " has borrowed the book: " + book.getTitle());
                book.decrementCopies(); // Update available copies
            } else {
                System.out.println("The book " + book.getTitle() + " is not available");
            }
        }
        // Method to return a book
        public void returnBook(BorrowedBooks borrowedBook) {
            if (borrowedBooks.contains(borrowedBook)) {
                borrowedBook.setReturnDate(new Date());
                borrowedBooks.remove(borrowedBook);
                System.out.println(getName() + " returned the book: " + borrowedBook.getTitle());
            } else {
                System.out.println("This book was not borrowed by " + getName());
            }
        }
        // Get list of borrowed books
        public List<BorrowedBooks> getBorrowedBooks() {
            return borrowedBooks;
        }
    }
```

The Customer class extends the User class to represent a library customer who can borrow and return books. It has a list to track borrowed books and offers methods for borrowing and returning books. When borrowing, the class checks if the book is available, creates a BorrowedBooks instance, and updates the book's available copies. When returning a book, the class updates the return date and removes the book from the borrowed list. This class ensures that customers can interact with the library's inventory, borrowing books and returning them efficiently while keeping track of their borrowed items.

1. Main class (Library)

```java
import java.util.ArrayList;
import java.util.Date;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Scanner;

public class Library {
    private static List<books> bookList = new ArrayList<>();
    private static Scanner scanner = new Scanner(System.in);
    private static List<Admin> admins = new ArrayList<>();
    private static List<User> users = new ArrayList<>();
    private static List<BorrowedBooks> borrowedBooksList = new ArrayList<>();
    private static List<BoughtBook> boughtBooksList = new ArrayList<>();
```

Java.util.ArrayList Used to create dynamic lists such as bookList, admins, and users Enables easy addition and removal of elements

Java.util.Scanner: Used for user input through the interface

java.util.Date:
Used to handle dates, such as the borrowing or returning date of books.

java.util.HashMap:
Used to store the number of times books are borrowed or bought to identify the most popular books.

java.util.List:
Represents an interface used to define lists like bookList, making the code more flexible.

java.util.Map:
An interface used with the HashMapobject to store key-value relationships, such as the book name and the number of times it has been borrowed.

```java
public static void main(String[] args) {
    initializeBooks();
    initializeAdmins();

    System.out.println("Welcome to the Library Management System");

    boolean running = true;
    while (running) {
        System.out.println("\n1. Login");
        System.out.println("2. Sign Up");
        System.out.println("3. Exit");
        System.out.print("Please select an option: ");
        int option = scanner.nextInt();
        scanner.nextLine(); // Consume newline

        switch (option) {
            case 1:
                login();
                break;
            case 2:
                signUp();
                break;
            case 3:
                running = false;
                System.out.println("Exiting...");
```

main():
The entry point of the application. Default data for books and admins is initialized using initializeBooks and initializeAdmins. The main menu displays options for login, sign-up, or exit. Based on the user's choice, the appropriate method for login or sign-up is called.

```java
private static void initializeBooks() {
    for (int i = 1; i <= 50; i++) {
        books book = new books("Book Title " + i, "Author " + i, "Publisher " + i, "ISBN" + i, 5, 10.0);
        bookList.add(book);
    }
}

private static void initializeAdmins() {
    admins.add(new Admin("Amal Alghamdi", "1234", "amal@gmail.com","5432",20));
    admins.add(new Admin("Haneen Abdulaziz","6491", "haneen@gmail.com","0751",20));
    admins.add(new Admin("Aqelah Alawi", "3015", "aqelah@gmail.com","5276",20));
    admins.add(new Admin("Shahad Adel", "7289", "shahad_a@gmail.com","9616",20));
}
private static void login() {
    System.out.print("Enter your ID: ");
    String id = scanner.nextLine();

    for (Admin admin : admins) {
        if (admin.getId().equals(id)) {
            System.out.println("\nWelcome, " + admin.getName() + "!");
            adminMenu(admin);
            return;
        }
    }
```

initializeBooks():
Creates a list containing 50 virtual books with different titles, authors, and publishers, then adds them to the bookList.

initializeAdmins():
Adds predefined admins with fixed names and IDs to the admins list.

login():
Prompts the user to enter an ID and searches for it in the admins and users lists. If the ID is found, it opens the appropriate menu using adminMenu if it's an admin or userMenu if it's a user. If the ID is not found, the user is directed to sign up.

```java
        for (User user : users) {
            if (user.getId().equals(id)) {
                System.out.println("\nWelcome, " + user.getName() + "!");
                userMenu(user);
                return;
            }
        }

        System.out.println("You are not registered. Please sign up.");
    }

    private static void signUp() {
    System.out.print("Enter your name: ");
    String name = scanner.nextLine();
    System.out.print("Enter your ID: ");
    String id = scanner.nextLine();
    System.out.print("Enter your email: ");
    String email = scanner.nextLine();
    System.out.print("Enter your phone number: ");
    String phoneNumber = scanner.nextLine();

    int age = 0;
    while (true) {
        System.out.print("Enter your age: ");
        try {
```

signUp():
Allows the user to create a new account. It prompts for personal information (name, email, phone number, age) and lets the user choose the account type (user or admin). The new account is then added to the appropriate list (users or admins).

```java
    }
}

    private static void userMenu(User user) {
        boolean userRunning = true;

        while (userRunning) {
            System.out.println("\nUser Menu");
            System.out.println("1. Borrow Book");
            System.out.println("2. Buy Book");
            System.out.println("3. Return Book");
            System.out.println("4. Display All Books");
            System.out.println("5. View Most Borrowed/Bought Books");
            System.out.println("6. Exit to Main Menu");
            System.out.print("Choose an option: ");
            int choice = scanner.nextInt();
            scanner.nextLine();

            switch (choice) {
                case 1:
                    borrowBook(user);
                    break;
                case 2:
                    BuyBook(user);
                    break;
```

userMenu(User user):
Displays a menu of options for users, such as borrowing a book, buying a book, returning a book, displaying all books, and viewing the most borrowed or bought books. Based on the user's choice, the appropriate method is called to execute the operation.

```
                case 3:
                    returnBook();
                    break;
                case 4:
                    displayBooks();
                    break;
                case 5:
                    viewMostBorrowedBoughtBooks();
                    break;
                case 6:
                    userRunning = false;
                    break;
                default:
                    System.out.println("Invalid choice. Please try again.");
            }
        }
    }

    private static void adminMenu(Admin admin) {
    boolean adminRunning = true;

    while (adminRunning) {
        System.out.println("\nAdmin Menu");
        System.out.println("1. Add Book");
        System.out.println("2. Remove Book");
```

adminMenu(Admin admin):
Displays a menu of options for admins, such as adding a new book, deleting a book, updating a book's data, displaying all books, and viewing the most borrowed or bought books. The corresponding method is executed based on the admin's choice.

```
                break;
            default:
                System.out.println("Invalid choice. Please try again.");
        }
    }
}
private static void addBook(Admin admin) {

    System.out.print("Enter Title: ");
        String title = scanner.nextLine();
        System.out.print("Enter Author: ");
        String author = scanner.nextLine();
        System.out.print("Enter Publisher: ");
        String publisher = scanner.nextLine();
        System.out.print("Enter ISBN: ");
        String isbn = scanner.nextLine();

    int numOfCopies;
    while (true) {
        System.out.print("Enter Number of Copies: ");
        if (scanner.hasNextInt()) {
            numOfCopies = scanner.nextInt();
            if (numOfCopies > 0) {
                break;
            } else {
                System.out.println("Error: Number of copies must be greater than zero. Please try again.");
```

addBook(Admin admin):
Allows the admin to add a new book by entering the book details (title, author, publisher, ISBN, number of copies, price). A new book object is created using the admin.addBook method and added to the bookList.

```
        }
        scanner.nextLine();

        books newBook = admin.addBook(title, author, publisher, isbn, numOfCopies, price);
        bookList.add(newBook);
        System.out.println("Book added successfully!");
    }


    private static void removeBook(Admin admin) {
        System.out.print("Enter ISBN of the book to remove: ");
        String isbn = scanner.nextLine();

        books bookToRemove = null;
        for (books book : bookList) {
            if (book.getisbn().equals(isbn)) {
                bookToRemove = book;
                break;
            }
        }

        if (bookToRemove != null) {
            admin.removeBook(bookToRemove);
            bookList.remove(bookToRemove);
            System.out.println("Book removed successfully!");
```

removeBook(Admin admin):
Enables the admin to delete an existing book by entering its ISBN. The book is searched for in the bookList, and if found, it is deleted using the admin.removeBookmethod.

```
        } else {
            System.out.println("Book not found.");
        }
    }

    private static void updateBook(Admin admin) {
        System.out.print("Enter ISBN of the book to update: ");
        String isbn = scanner.nextLine();

        books bookToUpdate = null;
        for (books book : bookList) {
            if (book.getisbn().equals(isbn)) {
                bookToUpdate = book;
                break;
            }
        }

        if (bookToUpdate != null) {
            System.out.print("Enter New Title: ");
            String newTitle = scanner.nextLine();
            System.out.print("Enter New Author: ");
            String newAuthor = scanner.nextLine();
            System.out.print("Enter New Publisher: ");
            String newPublisher = scanner.nextLine();
            System.out.print("Enter New ISBN: ");
```

updateBook(Admin admin):
Allows the admin to update the data of an existing book. The ISBN of the book to be updated is entered, followed by the new data (title, author, publisher, number of copies, price). The book is updated using the admin.updateBook method.

```
            } else {
                System.out.println("Invalid selection. Please select 1 or 2.");
            }
        } catch (NumberFormatException e) {
            System.out.println("Error: Please enter a valid number.");
        }
    }

        double amount = bookToBuy.getprice();

        Payment payment;
        if (paymentOption == 1) {
            double cashReceived = 0;
             while (true) {
            System.out.print("Enter cash received: ");
            try {
                cashReceived = Double.parseDouble(scanner.nextLine());
                if (cashReceived >= amount) {
                    payment = new CashPayment(amount, cashReceived);
                    break;
                } else {
                    System.out.println("Error: Cash received must be at least the price of the book.");
                }
            } catch (NumberFormatException e) {
                System.out.println("Error: Please enter a valid amount.");
            }
        }
        } else {
            payment = new CardPayment(amount);
        }

        payment.processPayment();

        // Gather buyer's information

        String buyerName = user.getName();
        String id = user.getId();


        int buyerAge = 0;
        while (true) {
        System.out.print("Enter Buyer Age: ");
        try {
            buyerAge = Integer.parseInt(scanner.nextLine());
            if (buyerAge > 0) {
                break; // Valid age
            } else {
                System.out.println("Error: Age must be greater than zero. Please try again.");
            }
        } catch (NumberFormatException e) {
            System.out.println("Error: Please enter a valid number for age.");
```

BuyBook(User user):
Allows the user to purchase a book by entering its ISBN. The availability of the book is checked, and the user selects a payment method (cash or card). The purchase is recorded in the boughtBooksList, and the number of available copies is reduced.

```
        if (newPrice > 0) {
            break;
        } else {
            System.out.println("Error: Price must be greater than zero. Please try again.");
        }
    } else {
        System.out.println("Error: Please enter a valid number for 'Price'.");
        scanner.next();
    }
}
scanner.nextLine();

        admin.updateBook(bookToUpdate, newTitle, newAuthor, newPublisher, newIsbn, newNumOfCopies, newPrice);
        System.out.println("Book updated successfully!");
    } else {
        System.out.println("Book not found.");
    }
}

private static void borrowBook(User user) {
System.out.print("Enter ISBN of the book to borrow: ");
String isbn = scanner.nextLine();

books bookToBorrow = null;
```

```java
    for (books book : bookList) {
        if (book.getisbn().equals(isbn) && book.Availability()) {
            bookToBorrow = book;
            break;
        }
    }

    if (bookToBorrow != null) {
        bookToBorrow.decrementCopies();
        BorrowedBooks borrowedBook = new BorrowedBooks(
            bookToBorrow.getTitle(),
            bookToBorrow.getAuther(),
            bookToBorrow.getpublisher(),
            bookToBorrow.getisbn(),
            1,
            user.getName(),
            new Date(),
            null
        );
        borrowedBooksList.add(borrowedBook);
        System.out.println("You have successfully borrowed: " + bookToBorrow.getTitle());
        borrowedBook.display();
    } else {
        System.out.println("Book not available for borrowing.");
    }
```

borrowBook(User user):
Allows the user to borrow a book by entering its ISBN. The book's availability is checked, the number of available copies is reduced, and the book is added to the borrowedBooksList.

```java
    }
}


    private static void displayBooks() {
        System.out.println("\nDisplaying all books:");
        for (books book : bookList) {
            book.display();
            System.out.println("\n----------------------");
        }
    }

private static void returnBook() {
    System.out.print("Enter the title of the book to return: ");
    String title = scanner.nextLine();

    boolean found = false;

    for (BorrowedBooks borrowedBook : borrowedBooksList) {
        if (borrowedBook.getTitle().equalsIgnoreCase(title)) {
            found = true;

            // Check if the book is returned late
            Date currentDate = new Date();
            if (borrowedBook.lateReturn(currentDate)) {
```

returnBook():
Enables the user to return a borrowed book by entering its title. It checks if the book exists in the borrowedBooksList. If the return is delayed, a fine is calculated. The number of available copies in the bookList is updated.

displayBooks():
Displays all the available books in the library. It iterates through the bookList and shows the details of each book using the book.display method.

```
    }
}

    private static void viewMostBorrowedBoughtBooks() {
        Map<String, Integer> borrowedCount = new HashMap<>();
        Map<String, Integer> boughtCount = new HashMap<>();

        for (BorrowedBooks borrowedBook : borrowedBooksList) {
            String title = borrowedBook.getTitle();
            borrowedCount.put(title, borrowedCount.getOrDefault(title, 0) + 1);
        }

        for (BoughtBook boughtBook : boughtBooksList) {
            String title = boughtBook.getTitle();
            boughtCount.put(title, boughtCount.getOrDefault(title, 0) + 1);
        }

        String mostBorrowedTitle = borrowedCount.entrySet().stream()
            .max(Map.Entry.comparingByValue())
            .map(Map.Entry::getKey)
            .orElse("No borrowed books");
        int mostBorrowedCount = borrowedCount.getOrDefault(mostBorrowedTitle, 0);

        String mostBoughtTitle = boughtCount.entrySet().stream()
            .max(Map.Entry.comparingByValue())
            .map(Map.Entry::getKey)
```

viewMostBorrowedBoughtBooks():
Identifies and displays the most borrowed and bought books. A HashMap is used to track the number of times each book is borrowed or bought. The most popular books are found using the stream and maxfunctions, and the results are summarized for the user.

ATTACHMENTS:
Library management system code zip